```
"""
Explicit PII/PHI Detector with Masking Support
Detects PII using Presidio + spaCy, returns weighted scores, E-Score, and masked
text.
"""
import json
from dataclasses import dataclass, asdict
from datetime import datetime
from pathlib import Path
from presidio_analyzer import AnalyzerEngine
from presidio_analyzer.nlp_engine import NlpEngineProvider

# ============================================================================
# CONFIGURATION
# ============================================================================
SENSITIVITY_WEIGHTS = {
    # Direct Identifiers
    "US_SSN": 1.0,
    "US_PASSPORT": 1.0,
    "US_DRIVER_LICENSE": 1.0,
    "US_ITIN": 0.95,
    "US_BANK_NUMBER": 0.95,
    "CREDIT_CARD": 0.95,
    "IBAN_CODE": 0.95,
    "CRYPTO": 0.90,

    # Strong Identifiers
    "PERSON": 0.90,
    "EMAIL_ADDRESS": 0.85,
    "PHONE_NUMBER": 0.80,
    "IP_ADDRESS": 0.75,

    # Location
    "LOCATION": 0.60,
    "NRP": 0.50,

    # Temporal
    "DATE_TIME": 0.50,

    # Organization/Other
    "ORGANIZATION": 0.50,
    "URL": 0.40,

    # Medical (Presidio built-in)
    "MEDICAL_LICENSE": 0.90,
    "UK_NHS": 0.90,
    "SG_NRIC_FIN": 0.90,
    "AU_ABN": 0.85,
    "AU_ACN": 0.85,
    "AU_TFN": 0.95,
    "AU_MEDICARE": 0.90,
    "IN_PAN": 0.90,
    "IN_AADHAAR": 0.95,
    "IN_VEHICLE_REGISTRATION": 0.70,
    "IN_VOTER": 0.85,
    "IN_PASSPORT": 1.0,
}

DEFAULT_WEIGHT = 0.50

# ============================================================================
# DATA CLASSES
# ============================================================================
@dataclass
```

```
class DetectedEntity:
    text: str
    entity_type: str
    start: int
    end: int
    confidence: float
    weight: float
    weighted_score: float
    context: str

@dataclass
class PIIAnalysisResult:
    text_length: int
    timestamp: str
    entities: list[DetectedEntity]
    e_score: float
    original_text: str
    masked_text: str

    def to_dict(self) -> dict:
        return {
            "text_length": self.text_length,
            "timestamp": self.timestamp,
            "e_score": round(self.e_score, 4),
            "entity_count": len(self.entities),
            "entities": [asdict(e) for e in self.entities],
            "masked_text": self.masked_text,
        }

    def to_json(self, indent: int = 2) -> str:
        return json.dumps(self.to_dict(), indent=indent)

    def save(self, filepath: str | Path) -> None:
        filepath = Path(filepath)
        filepath.parent.mkdir(parents=True, exist_ok=True)
        filepath.write_text(self.to_json())

# ============================================================================
# DETECTOR
# ============================================================================
class PIIDetector:
    """Detects PII/PHI using Presidio built-in recognizers and calculates E-
Score."""

    def __init__(self, score_threshold: float = 0.3, context_window: int = 50):
        self.score_threshold = score_threshold
        self.context_window = context_window
        self.analyzer = self._create_analyzer()

    def _create_analyzer(self) -> AnalyzerEngine:
        nlp_engine = NlpEngineProvider(nlp_configuration={
            "nlp_engine_name": "spacy",
            "models": [{"lang_code": "en", "model_name": "en_core_web_lg"}]
        }).create_engine()

        return AnalyzerEngine(nlp_engine=nlp_engine, supported_languages=["en"])

    def _get_context(self, text: str, start: int, end: int) -> str:
        ctx_start = max(0, start - self.context_window)
        ctx_end = min(len(text), end + self.context_window)
        ctx = text[ctx_start:ctx_end]

        if ctx_start > 0:
            ctx = "..." + ctx
```

```
        if ctx_end < len(text):
            ctx = ctx + "..."

        return ctx

    def _calculate_e_score(self, entities: list[DetectedEntity]) -> float:
        if not entities:
            return 0.0

        scores = [e.weighted_score for e in entities]
        max_score = max(scores)
        avg_score = sum(scores) / len(scores)

        return min(0.6 * max_score + 0.4 * avg_score, 1.0)

    def _mask_text(self, text: str, entities: list[DetectedEntity]) -> str:
        """Replace detected entities with mask tokens."""
        if not entities:
            return text

        # Sort entities by start position in reverse order to avoid offset
issues
        sorted_entities = sorted(entities, key=lambda e: e.start, reverse=True)

        masked = text
        for entity in sorted_entities:
            mask_token = f"[{entity.entity_type}]"
            masked = masked[:entity.start] + mask_token + masked[entity.end:]

        return masked

    def analyze(self, text: str) -> PIIAnalysisResult:
        results = self.analyzer.analyze(
            text=text,
            language="en",
            score_threshold=self.score_threshold
        )

        entities = []
        for r in results:
            weight = SENSITIVITY_WEIGHTS.get(r.entity_type, DEFAULT_WEIGHT)
            entities.append(DetectedEntity(
                text=text[r.start:r.end],
                entity_type=r.entity_type,
                start=r.start,
                end=r.end,
                confidence=round(r.score, 4),
                weight=weight,
                weighted_score=round(r.score * weight, 4),
                context=self._get_context(text, r.start, r.end),
            ))

        entities.sort(key=lambda e: e.start)
        masked_text = self._mask_text(text, entities)

        return PIIAnalysisResult(
            text_length=len(text),
            timestamp=datetime.now().isoformat(),
            entities=entities,
            e_score=self._calculate_e_score(entities),
            original_text=text,
            masked_text=masked_text,
        )
```

```
# ============================================================================
# PUBLIC API
# ============================================================================
def detect_pii(text: str, output_path: str | Path = None) -> PIIAnalysisResult:
    """
    Detect PII in text and optionally save to JSON.

    Args:
        text: Input text to analyze
        output_path: Optional path to save JSON results

    Returns:
        PIIAnalysisResult with entities, scores, E-Score, and masked text
    """
    detector = PIIDetector()
    result = detector.analyze(text)

    if output_path:
        result.save(output_path)

    return result

if __name__ == "__main__":
    sample = """
    Patient Jane Doe was seen at Boston Medical Center.
    Dr. Smith prescribed medication for hypertension.
    Contact: jane.doe@email.com, 617-555-1234. SSN: 489-36-8350.
    """

    result = detect_pii(sample, "e_result.json")
    Path("e_mask.txt").write_text(result.masked_text, encoding="utf-8")
```